

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Storage and Filesystems

Unit 7 • Linux (10-151-105)



Learning Outcomes

By the end of this unit, you will be able to manage disks on a Linux system. Specifically, you will:

- Create partitions to divide a disk into usable storage areas.
- Format filesystems to prepare partitions and logical volumes to hold data.
- Manage mount points, including persistent mounts via `/etc/fstab`.
- Build and extend Logical Volume Manager (LVM) storage.
- Monitor storage utilization to keep a disk from filling unexpectedly.



Where We're Headed This Week

1

Storage Foundations Media, storage models, and seeing your disks

2

Partitioning Dividing a disk — MBR vs GPT, fdisk and parted

3

Filesystems ext4, XFS, Btrfs, and formatting with mkfs

4

Mounting & Persistence mount, /etc/fstab, UUIDs, and mount options

5

LVM & Swap Pooling disks and growing storage live

6

Monitoring & Resilience df/du, fsck, and beyond one disk

LESSON 1

Storage Foundations



The Administrator's Storage Job

- Every directory you have used sits on real storage someone had to provision.
- Storage work means planning capacity, knowing the hardware, and using the right tools.
- It is high-stakes: a full root filesystem can crash a server, and a wrong command can erase data.
- The whole job reduces to one repeatable workflow, which is what this week teaches.

The add-storage workflow

1. Install the drive
2. Partition it
3. Add a filesystem
4. Mount it

...then make it persistent and monitor it.



Would an employer expect a junior admin to do this? Almost daily.

Storage Media and Interfaces

Media	Character
HDD	Spinning magnetic platters — inexpensive, large, relatively slow.
SSD	Flash memory — fast, more expensive per GB, shorter lifespan.
USB / external	Removable storage that connects over USB; treat as untrusted by default.

Interfaces vary by speed, cost, and scalability — SATA, SAS, and NVMe are common. An administrator balances speed, capacity, and cost for each role: a database server and an archive server have very different needs.



Match the storage to the workload, not the other way around.

Three Ways Systems Organize Data

Model	What it is and where it fits
File	Data as files in a directory hierarchy — what nearly every Linux server uses for local disks. Easy to use; scales out, not up.
Block	Raw blocks of capacity, common with storage area networks (SANs). Fast and efficient for large, unstructured data.
Object	Data as objects with metadata; very scalable and cheap, but awkward to modify — best for static data and backups.

Bottom line: a typical Linux server uses **file storage** on local disks, organized by an XFS or ext4 filesystem.

Seeing What's There

- **Storage devices live under /dev with the sd prefix:**
 - /dev/sda = first disk, /dev/sdb = second disk
 - /dev/sdb2 = second partition on the second disk
- In your pod, the system is on the first disk; the empty second disk is this week's workspace.

Command	Shows
lsblk	Disks, partitions, sizes, mount points (tree view)
blkid	Partition details — most importantly the UUID
hwinfo --short --storage	Hardware-level controller and disk detail
lsscsi -s	SCSI / NVMe devices with model and size

If a new disk does not appear, the cause is usually physical — a loose cable or bad slot. Check the connection before assuming a software problem.

LESSON 2

Partitioning



Why Partition a Drive?

- A raw disk is just capacity; partitioning divides it into usable sections, each able to hold its own filesystem.
- Administrators isolate directories that tend to fill onto their own partitions:
 - /home — users download endlessly
 - /var — logs can balloon when a service misbehaves
- If the partition holding / fills, the system can crash — isolating /home and /var protects it.



Partition design protects uptime

A filled log directory should fill its own partition — not the root filesystem the whole system depends on.

Two Partition-Table Formats: MBR vs GPT

	MBR (legacy)	GPT (modern)
Era	Older standard	Current; part of the UEFI standard
Max disk size	About 2 TB	Far beyond today's drives
Partitions	Four primary	Many, each with a unique ID
Resilience	Single boot record	Boot data stored in multiple places

Guidance: use GPT whenever possible. MBR survives only for compatibility with older systems.

Partitioning Tool: fdisk

fdisk is a menu-driven editor for a disk's partition table. Nothing is committed until you write.

Key	Action
m	Show the menu of commands
n	Create a new partition
d	Delete a partition
p	Print the current partition table
w	Write changes and exit
q	Quit WITHOUT saving

Typical session

```
fdisk /dev/sdb
n → p → +10G
p (review)
w (commit)

Prefer gdisk if you specifically want GPT, though
modern fdisk handles it too.
```

Partitioning Tool: parted

parted (GNU Parted) does the same job with named subcommands and is strong at creating and resizing partitions.

Subcommand	Action
mklabel gpt	Create a GPT partition table
mkpart	Create a partition (with type and size)
print	List the partition table
resizepart	Resize a partition
rm	Remove a partition
quit	Save changes and exit

- Recognizes and resizes NTFS partitions, too.
- gparted gives the same power in a GUI on desktop systems.
- Behaves identically on Rocky and Ubuntu.

Commit, Refresh, Confirm

- **After editing partitions, tell the kernel to re-read the table:**
 - `partprobe`
- **Then confirm the result:**
 - `cat /proc/partitions` — the system's current view
 - `lsblk` — the readable tree, with sizes
- `blkid` will now report a UUID for each new partition — you will need it next lesson.

LIVE DEMO

On the pod's second disk: create a GPT partition with `parted`, `partprobe`, then confirm with `lsblk`.

LESSON 3

Filesystems



Giving a Partition Structure

A partition is empty space until you write a filesystem onto it — the structure that tracks where files live, who owns them, and when they changed. Three you will meet on servers:

Filesystem	Notable traits
ext4	Long-standing default on many distros; journaled, well understood, broadly supported.
XFS	Modern Red Hat-family default; larger files and partitions, avoids inode exhaustion.
Btrfs	Highly scalable with built-in snapshots; default on some distros (Fedora, SUSE).

Creating a Filesystem: mkfs

You write any of them with the same command, just choosing the type:

```
mkfs.ext4 /dev/sdb2  
mkfs.xfs /dev/sdb2  
mkfs.btrfs /dev/sdb2  
( mkfs -t ext4 /dev/sdb2 works too )
```



CAUTION

mkfs OVERWRITES the partition, and the old data is effectively unrecoverable. Confirm the device first.

This destructive step is exactly why you double-check device names with lsblk before you press Enter.

Dual-Distro Divergence: Default Filesystem

	Rocky Linux 10 (Red Hat family)	Ubuntu Server 24.04 (Debian family)
Default root FS	XFS	ext4
Tools	mkfs, mount, fstab, LVM — identical	mkfs, mount, fstab, LVM — identical
This week	You choose the FS explicitly when you format the new disk	Same — the default never comes into play

This is the one real divergence in the unit — and it's only a default. The tooling is the same on both, so the skill transfers cleanly.

Two Special Filesystems to Recognize



tmpfs

Stores data in RAM rather than on disk — very fast, but gone at shutdown. Used for transient working space.



FUSE (Filesystem in Userspace)

Lets unprivileged programs present their own filesystems through a kernel bridge. Sandboxed apps use it for isolation — a small security win.

You won't create these by hand this week — the goal is to recognize them when you meet them.

LESSON 4

Mounting & Persistence



Mounting: Attaching Storage to the Tree

- A formatted partition isn't usable until you mount it — attach it to a directory in the filesystem tree.
- Linux has NO drive letters. Every partition appears somewhere inside the single directory tree.
- The directory you attach it to is called a mount point — normally an empty directory.

```
mkdir /projects  
mount /dev/sdb2 /projects  
umount /projects
```

Syntax reads like cp/mv: source first, destination second.

Spelling note: the command is `umount`, not `'unmount'`.

Gotcha: put files in a directory, then mount over it, and the originals vanish from view — they're still on the underlying disk and return when you unmount. That's why mount points are empty.

Testing the Mount

- **Confirm a mount worked and is writable two ways:**
 - Report on it: `df -h /projects` and `du -h /projects`
 - Prove it: `touch /projects/test` then `ls /projects`
- Removable media follows the same pattern, usually under `/media` or `/mnt`:
 - `mount /dev/sdc /media/usb`

df vs du

`df` (“disk free”) reports a whole filesystem — size, used, available, % full, mount point. `du` (“disk usage”) reports what the files inside a location consume — how you find what filled a disk.

Making Mounts Persistent: /etc/fstab

A mount made with the mount command disappears at reboot. To mount automatically at boot, record it in /etc/fstab, which the system reads during startup. Each line has six fields:

Field	Specifies
Block device	The partition — best given as a UUID
Mount point	The directory where it attaches
Filesystem type	xfs, ext4, etc.
Mount options	rw, ro, security options, and more
Dump	Legacy backup flag — almost always 0
fsck order	Boot-time check order: 1 root, 2 others, 0 skip

Example: `UUID=... /projects xfs defaults 0 2`

Why UUID — and Editing fstab Safely

- **Reference partitions by UUID, not device name:**
 - Device names (/dev/sdb2) can shift when disks are added or reordered.
 - A UUID is bound to the filesystem itself and stays correct. Get it with blkid.



Test before you reboot

A typo in /etc/fstab — a wrong UUID, missing mount point, bad option — can leave a filesystem unmounted or block the whole boot. After editing, run `mount -a` to mount everything in the file and catch mistakes while you can still fix them. If a bad entry does block boot, boot into rescue mode and correct it.

Mount Options: Control and Security

Option	Effect
rw / ro	Mount read-write (usual default) or read-only
noexec	Prevents running programs from this filesystem
nosuid	Ignores setuid/setgid — blocks privilege escalation
nodelv	Prohibits device files
noatime	Skips access-time updates — can improve performance
nofail	Boot even if the device is missing — good for removable

noexec, nosuid, and nodelv turn a data partition into a place where code can't run and privileges can't escalate — this is where storage and security meet.

Security Focus: The Disk Outlives the System



Storage is where data rests — and resting data is a target. A disk keeps its contents even when the system is off, so a stolen or discarded drive walks out with everything on it.

- **Mount options are access control.** noexec / nosuid / nodev on data and removable partitions.
- **Encryption protects data at rest.** LUKS makes a lost or stolen disk unreadable without the key.
- **Removable media is untrusted.** Auto-mounting a USB stick can attach a filesystem full of executables.
- **Utilization is a signal.** A disk filling unexpectedly is both an outage risk and worth investigating.

Why it matters: protecting a system isn't only about who can log in — it's about what happens to the data when the disk leaves your control.

LESSON 5

LVM & Swap



Swap Space and Virtual Memory

- RAM is finite. When memory runs low, Linux moves less-active data to a reserved disk area — swapping.
- That reserved area is swap space; combined with RAM it's called virtual memory.
- Payoff: the system doesn't simply run out of memory and crash. Cost: disk is far slower than RAM.
- Heavy, constant swapping is a signal — the server needs more RAM or a lighter workload.



Managing swap

`mkswap` create swap on a partition

`swapon -a` activate all swap

`swapoff -a` deactivate all swap

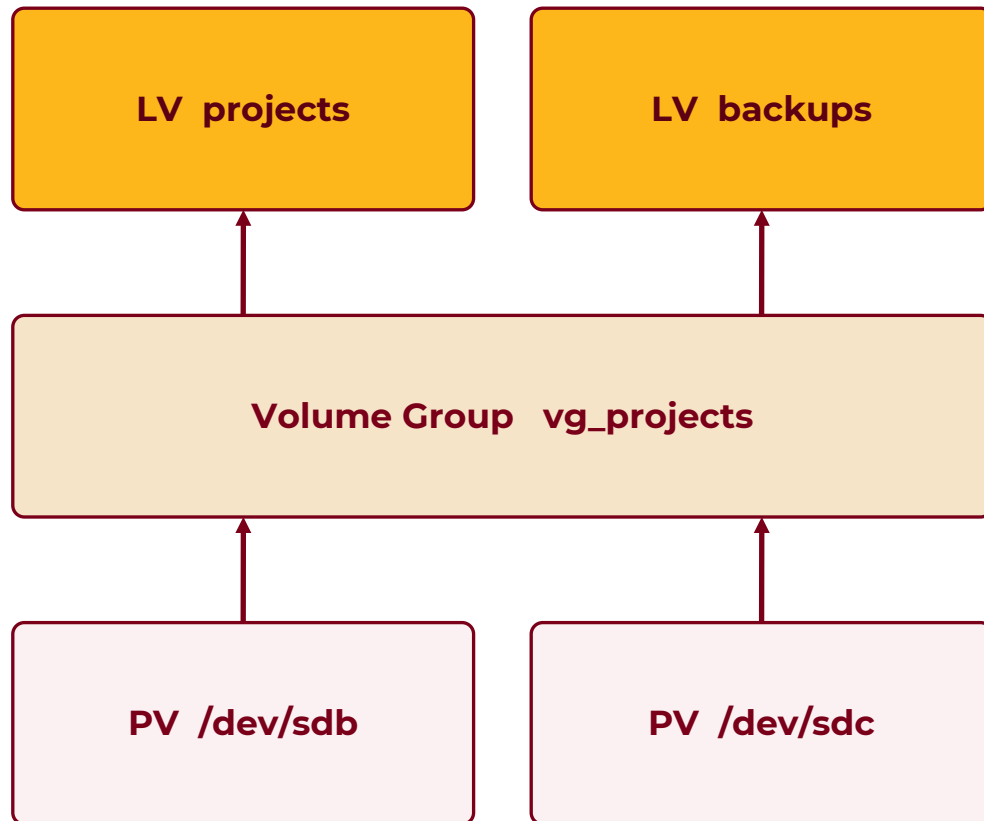
Your pod VMs already include swap from install.

Why LVM?

- Traditional partitioning treats every disk as a separate island — two 1 TB drives are two 1 TB spaces.
- The Logical Volume Manager pools capacity and hands it out flexibly — those drives become one 2 TB space.
- You can grow storage later, across multiple disks, without downtime.
- Many distributions install the root filesystem on LVM by default — including your pod's primary disk.

Flexibility is the whole point: provision conservatively now, grow on demand later — while users keep working.

The Three Layers of LVM



Physical Volume (PV)

A whole disk or partition given to LVM. `pvcreate`

Volume Group (VG)

One or more PVs pooled into one reservoir. `vgcreate`

Logical Volume (LV)

Space carved from a VG, used like a partition. `lvcreate`

Deploying an LVM Stack

Build it top to bottom through the layers — then it behaves like any partition:

```
pvcreate /dev/sdb /dev/sdc
vgcreate vg_projects /dev/sdb /dev/sdc
lvcreate --name projects --size 500G vg_projects
```

Confirm each layer with pvdisplay / vgdisplay / lvdisplay.

From here the steps are identical to a normal partition: add a filesystem with mkfs, create a mount point, and mount it. LVM volumes appear under /dev/mapper/ as {vg}-{lv}.

Growing Storage — Live

- **When an LV fills, add capacity without rebooting:**
 - pvcreate the new disk, then vgextend the volume group onto it
 - lvextend the logical volume into the new space
 - Grow the filesystem: xfs_growfs (XFS) or resize2fs (ext4)

Best practice: lvextend adds capacity to the volume, but does NOT grow the filesystem. You must run xfs_growfs or resize2fs afterward, or the new space stays invisible to users.

LIVE DEMO

Build a small LVM stack on the pod, then extend the LV and grow the filesystem — all live.

LESSON 6

Monitoring & Resilience



Monitoring Storage Utilization

Provisioning is half the job; watching capacity is the other half. A disk that quietly fills will eventually take down applications.

Command	Answers
df -h	How full is each mounted filesystem? (size, used, free, %)
du -h /home	What inside this location is consuming space?
lsblk	What's the device-and-partition layout and mount state?

A common first move when a filesystem is full: du -h on /home or /var/log to find the culprit fast.

The Paradox: Free Space, No New Files

- Sometimes `df` shows free space, yet the system says “no space left” when you create a file.
- Cause: inode exhaustion. Every file uses an inode (its metadata record).
- Filesystems like ext4 allocate a fixed number of inodes — millions of tiny files can use them all while bytes remain.
- **Confirm with `df -i` (watch the IUse% column); trace with `du --inodes`**
- XFS and Btrfs allocate inodes dynamically and rarely hit this.

Keeping Filesystems Healthy

- Filesystems can corrupt — most often from power loss or an ungraceful shutdown.
- fsck checks and repairs a filesystem, recovering damaged files into lost+found.
- **Critical rule: UNMOUNT the filesystem first — fsck on a mounted, in-use filesystem can worsen corruption.**
 - `umount /dev/sdb2` then `fsck -y /dev/sdb2`
- XFS uses its own tool, `xfs_repair`. If a storage fault blocks boot, fix it from rescue mode.

Each filesystem family also has tools for labeling, resizing, and reporting — `e2label` / `resize2fs` / `dumpe2fs` for ext4, `xfs_admin` / `xfs_growfs` / `xfs_info` for XFS.

Beyond One Disk: Encryption at Rest



LUKS2 (Linux Unified Key Setup)

The standard for full-drive encryption on Linux. Encrypted data is unreadable without the key, so a stolen or discarded disk doesn't leak its contents.

cryptsetup action	Does
luksFormat	Encrypt a device with LUKS2
luksOpen / luksClose	Unlock for use / lock again
luksAddKey	Associate additional key material

Awareness level this week — but this is the answer to “what if someone physically takes the drive?”

Beyond One Disk: RAID

A single disk is a single point of failure. RAID combines several drives for capacity, speed, fault tolerance — or a mix.

Level	Idea	Trade-off
RAID 0	Striping for speed	No redundancy — one disk fails, all data lost. 2+ disks.
RAID 1	Mirroring — identical copies	Full redundancy, fast reads, but half the usable space. 2+ disks.
RAID 5	Striping with parity	Speed plus rebuild after one failure. 3+ disks.

Built in hardware controllers (preferred) or software via mdadm (status in /proc/mdstat). Most admins choose RAID primarily for fault tolerance.

Beyond One Disk: Network Storage



Storage doesn't have to be local. Two appliance models share storage across the network.

	What it is
NAS	Network-Attached Storage — a file-sharing appliance over NFS (Linux-preferred) or SMB (Windows world, via Samba).
SAN	Storage Area Network — higher-end, faster, block storage over iSCSI or Fibre Channel, often with multipathing.

Disk quotas round out the toolkit — enabled via `usrquota/grpquota` in `/etc/fstab`, they cap how much space each user consumes so one person can't starve the rest.

This Week's Lab

NETLAB+ Lab: Storage and Filesystems

You'll bring the second disk online on both Rocky and Ubuntu:

- Confirm Linux sees the disk, partition it, and write a filesystem.
- Mount it, then make the mount persistent with a UUID-based `/etc/fstab` entry.
- Build an LVM stack (PV → VG → LV) and extend a logical volume.
- Verify everything with `lsblk`, `df`, and `du`.

Graded on the observable end state of each system — a working, verifiable configuration — not a description of what you did.



Key Takeaways

- **One workflow runs the whole unit: see the disk, partition, format, mount, persist, monitor.**
- Use GPT for partition tables; choose your filesystem deliberately (Rocky defaults XFS, Ubuntu ext4).
- mkfs is destructive — always confirm the device with lsblk first.
- Make mounts persistent with UUID-based /etc/fstab entries, and test with mount -a before rebooting.
- LVM pools disks and grows live — but remember to grow the filesystem after lvextend.
- Watch utilization with df and du; remember df -i for the inode paradox.
- Storage is a security boundary: mount options and encryption protect data at rest.



MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Questions?

Unit 7 • Storage and Filesystems

